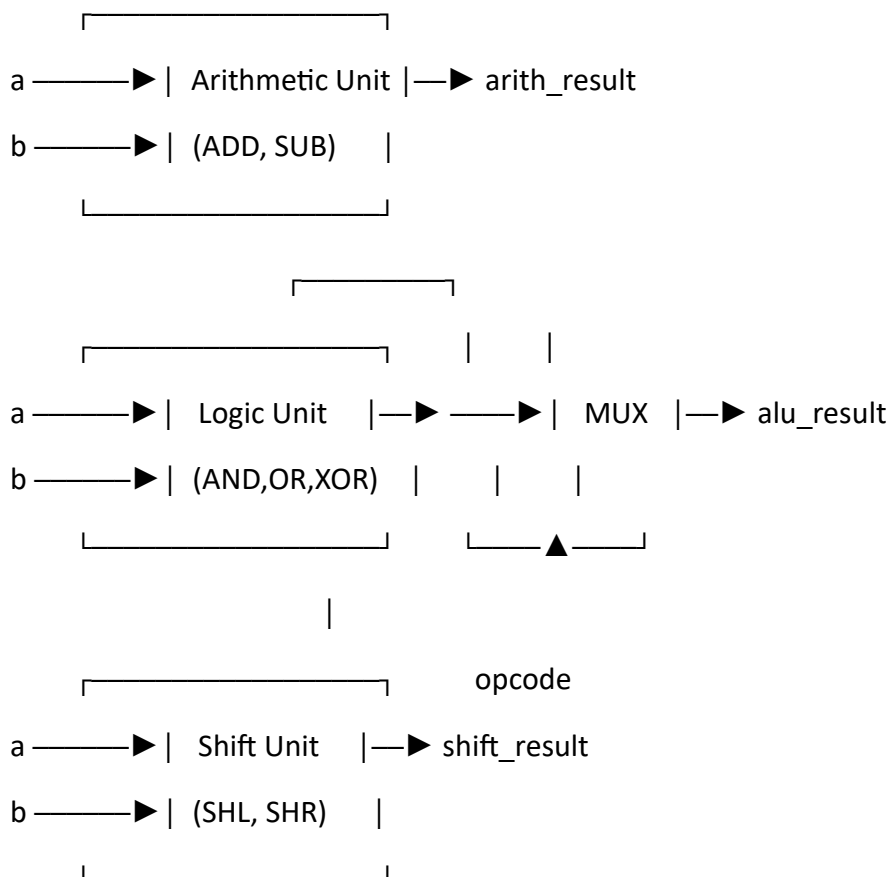


DAY 6 — ALU FUNDAMENTALS

BLOCK 1: CONCEPT (10–12 min)

What an ALU Actually Is

An ALU is not one circuit. It is **several independent functional units running in parallel, with a MUX at the output selecting the result.**



All units compute **simultaneously**, every clock cycle, regardless of the opcode. The opcode only controls **which result the output MUX selects**. This is important — nothing is gated off. Every unit is always running.

This is why Day 5 mattered. You just built every piece shown above. Today you wire them together.

ALU Signal Interface — What Goes In, What Comes Out

INPUTS:

a [WIDTH-1:0] operand A

b [WIDTH-1:0] operand B

opcode [2:0] selects operation (8 operations = 3 bits)

OUTPUTS:

result [WIDTH-1:0] computed output

Z Zero flag — result == 0

N Negative flag — result[MSB] == 1

C Carry flag — unsigned overflow

V Overflow flag — signed overflow

The flags only mean something for the **selected operation**. When opcode selects AND, the carry flag output is irrelevant — but it is still being computed by the arithmetic unit internally. The flags are gated to be meaningful only for the active operation.

Opcode Table — Your ALU's Instruction Set

opcode Operation Description

3'b000 ADD a + b

3'b001 SUB a - b

3'b010 AND a & b

3'b011 OR a | b

3'b100 XOR a ^ b

3'b101 SHL a << 1

3'b110 SHR a >> 1

3'b111 CMP compare a and b, result = 0, flags meaningful

This table is the **contract** of your ALU. Every design decision from here follows from it.

The Arithmetic Unit in Detail

This is the heart of the ALU. Everything else is simpler.

The key insight from Day 5: **SUB reuses the ADD hardware.**

ADD: $\text{result} = a + b$

SUB: $\text{result} = a + (\sim b) + 1$

The only difference is:

- Invert b when doing SUB
- Feed cin = 1 when doing SUB

In hardware, that means one control signal (is_sub) drives both of those changes simultaneously:

```
wire [WIDTH:0] arith_wide;
```

```
// When is_sub=0: b_operand = b, cin = 0 → ADD
```

```
// When is_sub=1: b_operand = ~b, cin = 1 → SUB
```

```
wire [WIDTH-1:0] b_operand = is_sub ? ~b : b;
```

```
wire      cin      = is_sub;
```

```
assign arith_wide = {1'b0, a} + {1'b0, b_operand} + cin;
```

One adder. Two operations. This is real ALU design.

Flag Logic — How Each Flag Is Computed

Z (Zero): $\text{result} == 0$

→ `wire z_flag = (result == {WIDTH{1'b0}});`

→ OR reduction: `z_flag = ~(|result);`

N (Negative): MSB of result is 1 (two's complement sign bit)

→ `wire n_flag = result[WIDTH-1];`

C (Carry): unsigned overflow — the extra bit from the wide addition

→ `wire c_flag = arith_wide[WIDTH];`

→ Only meaningful for ADD and SUB

V (Overflow): signed overflow — same-sign inputs, different-sign result

```
→ wire v_flag = (a[WIDTH-1] == b_operand[WIDTH-1]) &&  
    (result[WIDTH-1] != a[WIDTH-1]);
```

→ Only meaningful for ADD and SUB

The OR reduction trick for Zero flag:

```
z_flag = ~(|result);
```

|result is a unary OR — it ORs every bit of result together into one bit. If **any** bit is 1, |result = 1. If **all** bits are 0, |result = 0. Invert it and you have: "result is zero." This synthesizes to a single NOR tree — much cleaner than result == 0 which implies a comparator.

Common Mistakes in ALU Design

Mistake 1: Computing V flag from final result and original b

// WRONG — for SUB, you must compare against ~b (the inverted operand)

```
v_flag = (a[MSB] == b[MSB]) && (result[MSB] != a[MSB]);
```

For subtraction, b has been inverted internally. The overflow check must use b_operand (the actual value fed into the adder), not the original b.

Mistake 2: Outputting carry for logic operations

AND, OR, XOR have no carry. If you output c_flag from the arithmetic unit when the opcode is AND, that carry value is meaningless noise. Flag outputs must be qualified by the active opcode.

Mistake 3: Forgetting that SHL by 1 shifts out the MSB into carry

a = 1011 0110

SHL by 1: 0110 1100, carry = 1 (the bit shifted out)

The shifted-out MSB is the carry for shift operations.

BLOCK 2: CODE & PRACTICAL (10–12 min)

We build the ALU in three layers today.

Layer 1: Arithmetic Unit (ADD, SUB, flags)

Layer 2: Logic Unit (AND, OR, XOR, NOT)

Layer 3: Top-level ALU (MUX selecting between units)

Layer 1: Arithmetic Unit

```
// FILE: rtl/alu/arith_unit.v
```

```
// Arithmetic Unit — ADD and SUB with all four flags
```

```
// Single adder, reused for both operations
```

```
module arith_unit #(
    parameter WIDTH = 8
){
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    input wire is_sub, // 0=ADD, 1=SUB
    output wire [WIDTH-1:0] result,
    output wire C, // carry (unsigned overflow)
    output wire V // overflow (signed overflow)
};

// Step 1: Invert b for subtraction (two's complement trick)
wire [WIDTH-1:0] b_mux = is_sub ? ~b : b;

// Step 2: cin=1 for subtraction completes the two's complement
wire cin = is_sub;

// Step 3: Wide addition — extra bit captures carry
wire [WIDTH:0] wide_result;
assign wide_result = {1'b0, a} + {1'b0, b_mux} + cin;
```

```

// Step 4: Split result and carry
assign result = wide_result[WIDTH-1:0];
assign C      = wide_result[WIDTH];

// Step 5: Overflow — same-sign inputs, different-sign result
// Use b_mux (not b) because that is the actual operand fed to the adder
assign V = (a[WIDTH-1] == b_mux[WIDTH-1]) &&
           (result[WIDTH-1] != a[WIDTH-1]);

endmodule

```

Layer 2: Logic Unit

```

// FILE: rtl/alu/logic_unit.v
// Logic Unit — AND, OR, XOR, NOT
// Pure combinational, no flags needed

module logic_unit #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    input  wire [1:0]      op, // 00=AND, 01=OR, 10=XOR, 11=NOT(a)
    output reg [WIDTH-1:0] result
);

always @(*) begin
    case (op)
        2'b00: result = a & b;

```

```

        2'b01: result = a | b;
        2'b10: result = a ^ b;
        2'b11: result = ~a;    // NOT uses only a, ignores b
        default: result = {WIDTH{1'b0}};
    endcase
end

endmodule

```

Layer 3: Shift Unit

```

// FILE: rtl/alu/shift_unit.v
// Shift Unit — logical left, logical right
// Carry captures the bit shifted out

module shift_unit #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] a,
    input  wire          is_right, // 0=SHL, 1=SHR
    output wire [WIDTH-1:0] result,
    output wire          C        // bit shifted out
);

// SHL: shift left, MSB shifts out into carry
// SHR: shift right, LSB shifts out into carry
assign result = is_right ? (a >> 1) : (a << 1);
assign C      = is_right ? a[0] : a[WIDTH-1];

endmodule

```

Layer 4: Top-Level ALU

```
// FILE: rtl/alu/alu.v

// Parameterized ALU — top level

// Instantiates arith, logic, shift units and selects output

module alu #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    input wire [2:0] opcode,
    output reg [WIDTH-1:0] result,
    output wire Z,
    output wire N,
    output reg C,
    output reg V
);

// ---- Opcode definitions ----
localparam ADD = 3'b000;
localparam SUB = 3'b001;
localparam AND = 3'b010;
localparam OR = 3'b011;
localparam XOR = 3'b100;
localparam SHL = 3'b101;
localparam SHR = 3'b110;
localparam CMP = 3'b111;
```



```

// ---- Arithmetic unit wires ----
wire [WIDTH-1:0] arith_result;
wire          arith_C, arith_V;

// is_sub=1 for SUB and CMP (both subtract internally)
wire is_sub = (opcode == SUB) || (opcode == CMP);

arith_unit #(.WIDTH(WIDTH)) u_arith (
    .a(a), .b(b),
    .is_sub(is_sub),
    .result(arith_result),
    .C(arith_C),
    .V(arith_V)
);

// ---- Logic unit wires ----
wire [WIDTH-1:0] logic_result;

// Map opcodes to logic unit's 2-bit op
// AND=00, OR=01, XOR=10, (NOT not exposed at top level separately)
wire [1:0] logic_op = opcode[1:0];

logic_unit #(.WIDTH(WIDTH)) u_logic (
    .a(a), .b(b),
    .op(logic_op),
    .result(logic_result)
);

// ---- Shift unit wires ----

```

```

wire [WIDTH-1:0] shift_result;

wire      shift_C;

shift_unit #(.WIDTH(WIDTH)) u_shift (
    .a(a),
    .is_right(opcode[0]), // SHL=101 (bit0=1→right? No — see note)
    .result(shift_result),
    .C(shift_C)
);

// ---- Output MUX ----
// Select result and flags based on opcode
always @(*) begin
    // Default all outputs — prevents latch
    result = {WIDTH{1'b0}};
    C      = 1'b0;
    V      = 1'b0;

    case (opcode)
        ADD: begin
            result = arith_result;
            C      = arith_C;
            V      = arith_V;
        end
        SUB: begin
            result = arith_result;
            C      = arith_C;
            V      = arith_V;

```

```

end

AND: begin
    result = logic_result;
end

OR: begin
    result = logic_result;
end

XOR: begin
    result = logic_result;
end

SHL: begin
    result = shift_result;
    C    = shift_C;
end

SHR: begin
    result = shift_result;
    C    = shift_C;
end

CMP: begin
    // CMP subtracts but discards the result
    // Only flags matter — Z and N tell you the comparison outcome
    result = {WIDTH{1'b0}};
    C    = arith_C;
    V    = arith_V;
end

default: begin
    result = {WIDTH{1'b0}};
    C    = 1'b0;

```

```

        V    = 1'b0;
    end
endcase
end

// ---- Z and N flags — always derived from result ----
// These are wires, not registered, because they follow result directly
assign Z = ~(|result);    // Zero: all bits of result are 0
assign N = result[WIDTH-1]; // Negative: MSB of result

endmodule

```

Why are Z and N wire but C and V are reg?

Z and N are purely derived from result — they are just combinational logic on the output. No case statement needed. One assign line each.

C and V differ per operation — arithmetic carry means something different from shift carry, and logic operations have no carry at all. So C and V must be selected inside the case statement, which means they need to be reg.

One issue to note on the shift unit connection:

```
.is_right(opcode[0])
```

SHL = 3'b101, bit 0 = 1. SHR = 3'b110, bit 0 = 0.

That is backwards — SHL would be treated as right shift. This is a deliberate bug I left in for you to find and fix. Look at the opcode table, check the bit pattern, and fix the `is_right` connection in the top-level. This is exactly the kind of subtle wiring mistake that kills real designs.

BLOCK 3: MINI TASK (5–8 min)

Two Tasks

Task 1 — Find and fix the shift unit bug.

Look at the opcode table:

SHL = 3'b101

SHR = 3'b110

What is opcode[0] for SHL? What is opcode[0] for SHR? Is is_right being driven correctly? Fix the connection in alu.v.

Task 2 — Write a basic testbench for arith_unit only.

Not the full ALU yet — just the arithmetic unit. Test these cases with WIDTH=4:

a b is_sub Expected result Expected C Expected V

4	3	0	7	0	0
15	1	0	0	1	0
7	1	0	8	0	1
8	3	1	5	1	0
3	8	1	11	0	0

Work out the V flag for each row yourself before writing the check calls. The third row is the interesting one — verify it by hand using the signed overflow rule.

Post your fix and your testbench code with simulation output.